

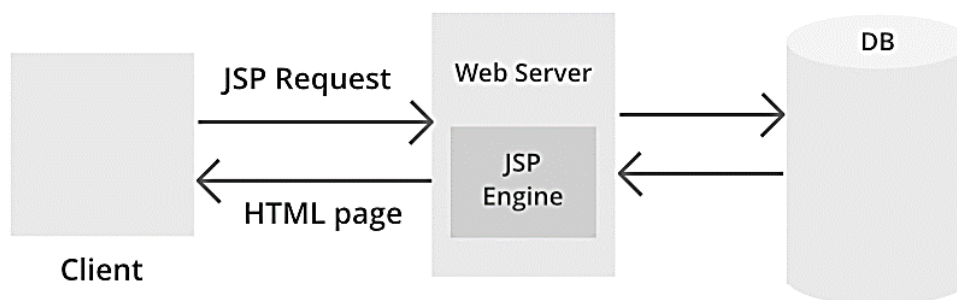
UNIT 4: Java Server Pages and JSTL

JSP Architecture and Lifecycle, Action Elements, Implicit Objects, Scripting Elements, Scope and EL (Expression Language), JSP Standard Tag Libraries (JSTL), Developing Dynamic Web Applications with JSP.

JSP Architecture and Lifecycle

JSP Architecture

JSP (Java Server Pages) uses a three-tier architecture with a client, web server, and database. When the client sends a request, the web server's JSP engine processes the JSP file by converting it into a servlet, compiling, and executing it. The generated HTML is sent back to the client. JSP container is responsible for intercepting requests for JSP pages. The following diagram shows the architecture of JSP.



Architecture of JSP

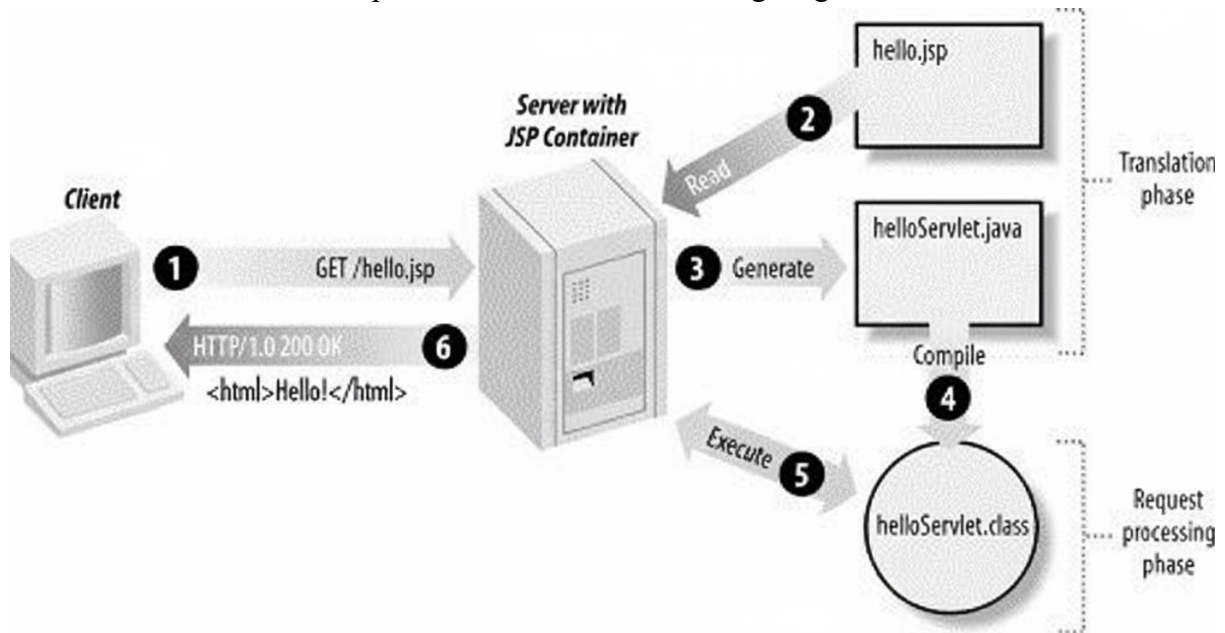
JSP Processing

The following steps explain how the web server creates the Webpage using JSP –

1. As with a normal page, your browser sends an HTTP request to the web server.
2. The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with .jsp instead of .html.
3. The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to `println()` statements and all JSP elements are converted to Java code. This code implements the corresponding dynamic behavior of the page.
4. The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.

5. A part of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. The output is further passed on to the web server by the servlet engine inside an HTTP response.
6. The web server forwards the HTTP response to your browser in terms of static HTML content.
7. Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page.

All the above-mentioned steps can be seen in the following diagram –

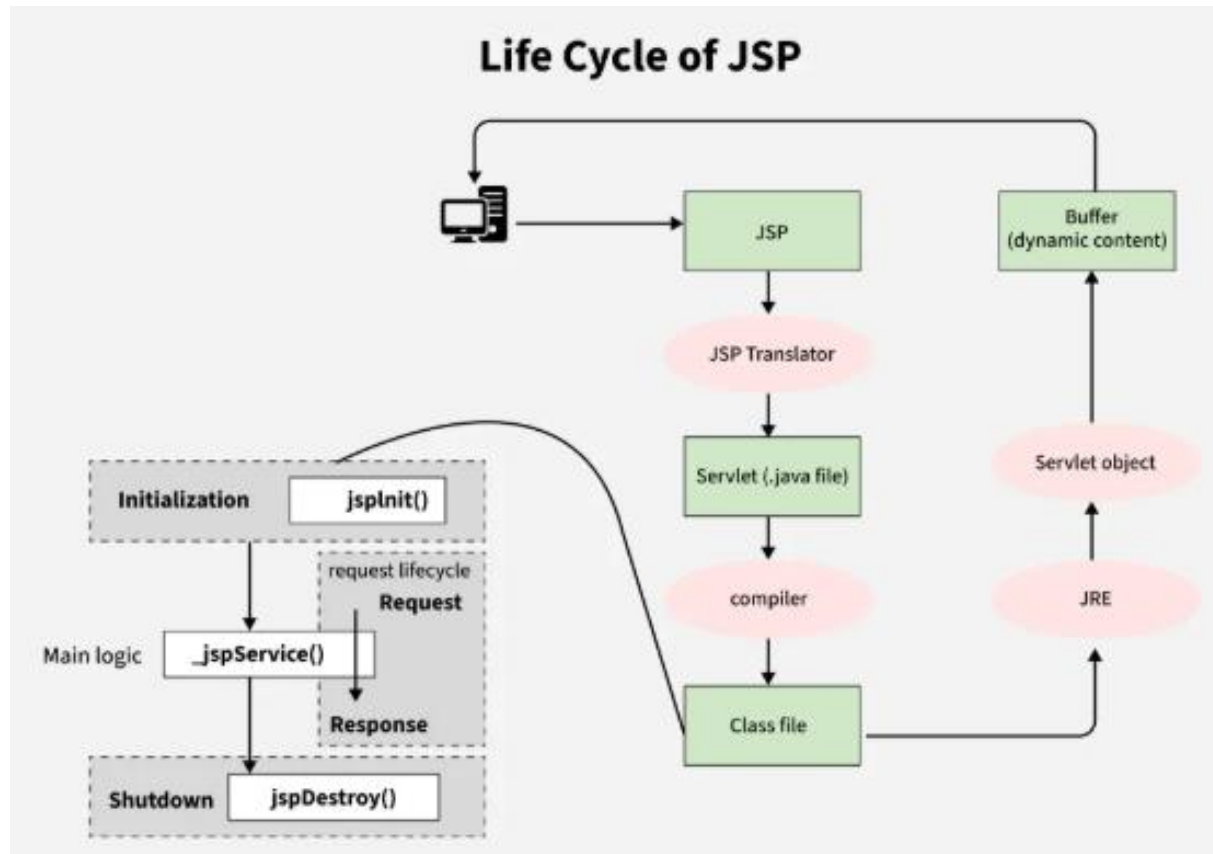


Life Cycle of JSP

The life cycle of a JavaServer Page (JSP) consists of various phases that start from its creation, followed by its translation into a servlet, and finally managed by the servlet lifecycle. The JSP engine handles this process automatically.

Steps of JSP Life Cycle

1. Translation of JSP page to Servlet
2. Compilation of JSP page(Compilation of JSP into test.java)
3. Classloading (test.java to test.class)
4. Instantiation(Object of the generated Servlet is created)
5. Initialization(jspInit() method is invoked by the container)
6. Request processing(_jspService() is invoked by the container)
7. JSP Cleanup (jspDestroy() method is invoked by the container)



1. Translation of JSP to Servlet

- The JSP file is parsed and converted into a Java servlet source file (test.java).
- This step checks for syntax correctness.

```
public class TestServlet extends HttpServlet
{
}
```

2. Compilation of JSP page

- The generated test.java file is compiled into a class file (test.class).
- This step converts the servlet code into bytecode.

```
public class TestServlet extends HttpServlet
{
}
```

3. Classloading

- The container dynamically loads the compiled class.

4. Instantiation

- The container creates an instance of the generated servlet class.
- This instance handles multiple requests unless explicitly configured otherwise.

```
TestServlet servlet = new TestServlet();
```

5. Initialization (jspInit())

- This method is called only once when the JSP is first loaded.
- It is used for initializing resources like database connections or configurations.

```
public void jspInit() {
```



```
// Initialization code, like setting up resources.
System.out.println("JSP Initialized.");
}
```

6. Request Processing (_jspService())

- This method is called for every request.
- It cannot be overridden because it is auto-generated and declared as final.
- It receives HttpServletRequest and HttpServletResponse objects to handle the request.

```
public void _jspService(HttpServletRequest request, HttpServletResponse response) {
    response.getWriter().write("<html><body>Hello, World!</body></html>");
}
```

7. JSP Cleanup (jspDestroy())

- This method is called once before removing the JSP from service.
- It is used for releasing resources, such as closing database connections or cleaning up open files.

```
public void jspDestroy() {
    System.out.println("JSP Destroyed.");
}
```

This Lifecycle ensures that JSP pages are efficiently compiled, managed and cleaned up by the server container.

JSP- Action Elements (Action Tags)

JSP action tags are special elements that give instructions to the JSP container for handling various runtime tasks. They execute on the server side and help developers include external resources, forward requests, and interact with JavaBeans or plugins. They are always enclosed within `<jsp: ... >` and can be placed anywhere in the JSP page where dynamic behavior is required.

Common JSP Action Tags

1. `<jsp:include>` Tag

Includes content from another resource (JSP, HTML, or servlet) into the current page at request time.

Syntax:

```
<jsp:include page="filename.jsp" />
```

Usage: Helps in modularizing JSP pages by reusing header, footer, or navigation components.

2. `<jsp:forward>` Tag

Forwards the current request to another resource such as a JSP, HTML, or servlet. The client is unaware of the forwarding.

Syntax:

```
<jsp:forward page="destination.jsp" />
```

Note: Once the forward tag is executed, the rest of the current page is skipped.



3. <jsp:useBean> Tag

Instantiates or locates an existing JavaBean to use within a JSP page.

Syntax:

```
<jsp:useBean id="beanId" class="packageName.ClassName" />
```

Example:

```
<jsp:useBean id="user" class="com.example.User" />
```

Note: If the bean does not exist in the specified scope, a new instance is automatically created.

4. <jsp:setProperty> Tag

Sets a property value of a JavaBean component.

Syntax:

```
<jsp:setProperty name="beanName" property="propertyName" value="propertyValue" />
```

Example:

```
<jsp:setProperty name="user" property="username" value="John" />
```

Note: It can also retrieve values directly from request parameters by using property="*".

5. <jsp:getProperty> Tag

Retrieves and displays the value of a JavaBean property.

Syntax:

```
<jsp:getProperty name="beanName" property="propertyName" />
```

Example:

```
<jsp:getProperty name="user" property="username" />
```

Usage: Commonly used to display data stored in JavaBeans directly on JSP pages.

6. <jsp:plugin> Tag

Generates HTML code for browser-specific plugins such as Java applets.

Syntax:

```
<jsp:plugin type="pluginType" code="pluginCode" />
```

Example:

```
<jsp:plugin type="applet" code="MyApplet.class" />
```

Note: This tag ensures that required browser plugins are loaded properly.

7. <jsp:attribute> Tag

Defines attribute values for custom JSP tags.

Syntax:

```
<jsp:attribute name="attributeName" value="attributeValue" />
```

Example:

```
<jsp:attribute name="color" value="blue" />
```

Usage: Used inside custom tag handlers to pass dynamic attributes.

8. <jsp:body> Tag

Defines the body content of a custom JSP tag.

Syntax:

```
<jsp:body> Body Content </jsp:body>
```

Example:

```
<jsp:body>
```

```
<p>This is the body content.</p>
```

```
</jsp:body>
```

JSP - Implicit Objects

Web containers create these implicit objects when they translate JSP pages to servlet. They are available to all JSP pages. They are written in scriptlet tags except declaration. It is due the fact that everything that is in the scriptlet tag goes to `_jspService()` method where processing of requests occurs, whereas declaration goes to source file generated outside `_jspService()` method. They get called directly and need not to be declared. They are also called in-built objects.

There are 9 implicit objects available in the web container. Out of these 9, 7 are objects while 2 are passed as parameters to the `_jspService()`. All the objects are:

1. out
2. request
3. response
4. config
5. session
6. application
7. page
8. pageContext
9. exception

1. out

It is the most used implicit object that comes from `java.servlet.jsp.JspWriter`. It's work is to write data into a buffer which would be sent to the client as output. It possesses the access to servlet output stream. Servlet uses `PrintWriter` but `JspWriter` has some additional features to it. `JspWriter` has options for buffer and I/O Exceptions which are not available in `PrintWriter`.

Syntax: `out.methodName();`

Out uses various methods such as:

a. void print(): It prints the data to the output. One can specify the data type too.

`out.print(datatype d);`

b. void println(): It prints the data type and terminates the line.

`out.println(datatype d);`

c. void flush(): The contents in the buffer(if present) gets written to the output and the stream is flushed.

`out.flush();`

d. void clear(): It clears the buffer. Even if something is present in the buffer, it doesn't get written to output. It will throw an exception, if this method is called on a buffer that is already flushed.

`out.clear();`

e. void clear buffer(): It is similar to clear but it doesn't give an exception if called on a flushed buffer.

`out.clear buffer();`

f. boolean isAutoFlush(): This method checks that the buffer is set to flush automatically or not.

`out.isAutoFlush();`



g. int getBufferSize(): This method returns the size of the buffer in bytes.

out.getBufferSize();

h. int getRemaining(): This method returns the remaining bytes of buffer that are empty before the buffer overflows.

out.getRemaining();

Example:

```
Out2.jsp
<html>
<head>
<title>IMPLICIT OBJECT</title>
</head>
<body>
<%
out.println("first statement");
out.println("second");
out.println("third");
%>
</body>
</html>
```

2. request

This is an in-built implicit object of `javax.servlet.http.HttpServletRequest`. It is passed as a parameter to `jspService` method. This instance is created each time a request is generated. Using this object we can request for a parameter session, cookies, header information, server name, server port, `contentType`, character encoding etc. It has following methods:

Syntax: `request.methodname("value");`

Methods under request parameter are:

a. getParameter(): This method is used so that the value of request can be got.

`String obj=request.getParameter("name");`

b. getParameterNames(): It gives the Enumeration of requested values.

c. getParameterValues(String name): It returns a whole array of parameters.

d. getAttribute(String name): This method gets the value of the attributes present.

`request.getAttribute("value");`

e. getAttributeNames(): This method is used to generate all the attributes that are present in that session.

f. setAttribute(String, Object): This method sets the value of an attribute according to user requirements. For example, If we want to set password value as admin then,

`str=admin`

`String str=request.setAttribute("Password", str)`

This will set admin as password.

g. removeAttribute(str): This method will remove the mentioned attribute from that JSP page.

`request.removeAttribute("value");`

h. getCookies(): This method of request will give an array of cookie objects used under cookie handling.



- i. getHeader(String name):** This method will get the information regarding the header of the request.
- j. getHeaderNames():** This method will return an enumeration of all the header names available.
- k. getRequestedURI():** This method will return the Current JSP page's URL to the user.
- l. getMethod():** This method will return the method used for the request. For example, GET for getMethod(), POST for postMethod().
- m. getQueryString():** This method will return the string in the url after the question mark for the associated page.

Example:

```
form.html
<html>
<head><title>My login</title>
</head>
<body>
<form action="user.jsp">
<a>Username:</a><input type="text" name="username"> <br/>
<a>Password:</a><input type="text" name="password">
<input type="submit" value="go"><br/>
</form>
</body>
</html>

user.jsp
<html>
<head><title>login</title></head><body>
<%
String name=request.getParameter("username");
String pass=request.getParameter("password");
out.print("Welcome user, you entered your name as "+name);
%>
</body>
</html>
```

3. response

This is an object from HttpServletResponse. This object rather gets passed as a parameter to jspService() like request. It is the response given to the user. One can redirect, get header information, add cookies, send error messages using this object.

Syntax: response.nethodName();

Methods used with response object are:

- a. void setContentType(String type):** This method sets the contempt type of browser and browser interprets according to it.
 response.setContentType("text/html");
 response.setContentType("image/gif");



b. void sendRedirect(String address): This method will redirect users to other destinations.

```
response.sendRedirect("http://www.google.com");
```

c. void addHeader(String name, String value): This method adds header to the response

```
response.addHeader("Address", "Google.com");
```

d. void setHeader(String name, String value): This method will set the header value

```
response.setHeader("Address", "G.com");
```

e. boolean containsHeader(String name): It checks that if header is present or not. It can have 2 values either true/false

```
response.containsHeader("Address");
```

f. void addCookie(Cookie value): Response will contain cookie using this method.

```
response.addCookie(Cookie kk);
```

g. void sendError(int status_code, String message): Error response is sent using this method.

```
response.sendError(404, "error: Page was not found ");
```

Example:

redirect.html

```
<html>
```

```
<head><title>My login</title>
```

```
</head>
```

```
<body>
```

```
<form action="user2.jsp">
```

```
<a>Username:</a><input type="text" name="username"> <br/>
```

```
<a>Password:</a><input type="text" name="password">
```

```
<input type="submit" value="go"><br/>
```

```
</form>
```

```
</body>
```

```
</html>
```

user2.jsp

```
<html>
```

```
<head><title>login</title></head>
```

```
<body>
```

```
<%
```

```
response.sendRedirect("http://www.google.com");
```

```
%>
```

```
</body>
```

```
</html>
```

4. config

It is an object of `javax.servlet.ServletConfig`. It is used to get the information regarding configuration of JSP pages. For example get Servlet Name etc. It gives out the parameters related to servlet mapping initialization.

Syntax: `config.methodName();`

Methods used with config object are:



1. **String getInitParameter(String name)** – This method gets the parameter name.
 2. **Enumeration getInitParameterNames()** – This method will return an enumeration of initialization params.
 3. **getServletContext()** – It will return reference to Servlet context.
 4. **String getServletName()** – It returns the name of the servlet which was defined in the web.xml file used for mapping.
- String s = config.getServletName();

Example:

```

<html>
<head><title>Config</title>
</head>
<body>
<form action=.jsp">
<a>Username:</a><input type="text" name="username"> <br/>
<input type="submit" value="go"><br/>
</form>
</body>
</html>
web.xml
<web-app>
<servlet>
<servlet-name>sample</servlet-name>
<jsp-file>/one.jsp</jsp-file>
<init-param>
<param-name>drivername</param-name>
<param-value>abc.JdbcOdbcDriver</param-value>
</init-param>
</servlet>
<servlet-mapping>
<servlet-name>sample</servlet-name>
<url-pattern>/one</url-pattern>
</servlet-mapping>
</web-app>
one.jsp
<html>
<head><title>Config2</title></head>
<body>
<%
out.print("Welcome "+request.getParameter("username"));
String driver=config.getInitParameter("drivername");
out.print("driver name is="+driver);
%>
</body>
</html>

```



5. session

session object is the implicit object under `javax.servlet.http.HttpSession`. It is used to get session information as it tracks sessions between client requests. It can be used to get, set as well as remove under scope of session. As session creation and management is a cumbersome project so if you don't want a session to be created we can set the session in page directive as false.

Syntax: `session.getAttribute()`

Methods used with session object are:

- a. `setAttribute(String, object)`** – This method will assign a string to the object and save it in session.
- b. `getAttribute(String name)`** – This method gets the object that is set by `setAttribute`.
- c. `removeAttribute(String name)`** – Objects that are in the session can be removed using this method.
- d. `getAttributeNames`** – It returns enumeration of the objects in session.
- e. `getCreationTime`** – This method returns time when the session was in active mode.
- f. `getId`** – This method gives out the unique id of the session.
- g. `isNew`** – This method checks if a session is new or not. It would return true if cookies are not there.
- h. `getMaxInactiveInterval`** – Returns time span, the session was inactive.
- i. `getLastAccessedTime`** – This method gives the last accessed time of a session.

Example:

```
<html>
<head><title>Config</title></head>
<body>
<form action="first.jsp">
<input type="text" name="username">
<input type="submit" value="go"><br/>
</form>
</body>
</html>
First.jsp
<html>
<head><title>Config</head></title>
<body>
<%
String myname=request.getParameter("username");
out.print("Welcome "+myname);
session.setAttribute("user",myname); %>
<a href="second.jsp">second jsp page</a>
</body>
</html>
second.jsp
<html>
<body>
<%
```



```
String name=(String)session.getAttribute("user");
out.print("Hello "+myname); %>
</body>
</html>
```

6. application

It is an implicit object of ServletContext from javax.servlet.ServletContext implementation. This object is created by the web container only when an application is deployed. It is generated one per application. As the name suggests, it interacts with servlet. It can get set or even remove the attributes under application scope. It can exclusively be used to get the object Request Dispatcher. Using Request Dispatcher, it forwards the request to other pages.

Syntax: application.methodName("value");

Methods available under application scope are:

a. Object getAttribute(String attributeName): This method gets the object available in the attribute and returns it.

```
String a = (String)application.getAttribute("Name");
```

b. void setAttribute(String attributeName, Object object): This method will set the value of attribute.

```
application.setAttribute("Address", "Value of Address")
```

c. void removeAttribute(String objectName): This method will remove an attribute from application.

```
application.removeAttribute("Name");
```

d. String getRealPath(String value): This method will return the absolute path of the file.

```
String abspath = application.getRealPath("/form.html");
```

e. void log(String message): This method stores logs to default log file of JSP Engine

```
application.log(" error 404 page not found");
```

f. String getServerInfo(): Returns name and version of web container

```
application.getServerInfo();
```

Example:

```
<html>
<head>
<title>Application object</title>
</head>
<body>
<a>This is an example for application implicit object</a>
<% application.getRealPath("/form.html"); %>
</body>
</html>
```

7. PageContext

It is the implicit object from javax.servlet.jsp.PageContext implementation where PageContext is an abstract implementation. This object has the capability to get, set and remove attributes that fall under 4 scopes—page, request, session and application scopes. This object has references to other implicit objects. It contains information of directives, buffer information, error page URL. It holds reference to request and response as well. This can support over 40 methods that are inherited from ContextClass. It is even used to get info of a complete JSP page.



Syntax: `pageContext.methodName("parameter");`

Some useful methods used with `pageContext`:

a. Object `getAttribute(String attributeName, int scope)`: This method finds/gets the attribute from the suggested scope

- `Object obj1 = pageContext.getAttribute("name", PageContext.SESSION_CONTEXT);`
- `Object obj2 = pageContext.getAttribute("address", PageContext.REQUEST_CONTEXT);`
- `Object obj3 = pageContext.getAttribute("number", PageContext.PAGE_CONTEXT);`
- `Object obj4 = pageContext.getAttribute("data", PageContext.APPLICATION_CONTEXT);`

b. void `setAttribute(String attributeName, Object attributeValue, int scope)`: This method will set the attribute in the scope along with its value.

`Object obj1 = pageContext.setAttribute("name", "Kajal", PageContext.SESSION_CONTEXT);`

c. void `removeAttribute(String attributeName, int scope)`: This method will remove the attribute mentioned from referred scope.

`Object obj1 = pageContext.removeAttribute("name", PageContext.SESSION_CONTEXT);`

d. Object `findAttribute(String attributeName)`: This method will find the attribute in all the four scopes and when found none, return null value. The difference between get and find is that scope is defined in the get method.

`Object obj2 = pageContext.findAttribute("address");`

Example:

`myindex.html`

```
<html>
<head><title>PageContext</title>
</head>
<body>
<form action="PageContext1.jsp">
<input type="text" name="username">
<input type="submit" value="go"><br/>
</form>
</body>
</html>
```

`PageContext1.jsp`

```
<html>
<head><title>PageContext</title></head>
<body>
<%
String myname=request.getParameter("username");
out.print("Welcome "+myname);
pageContext.setAttribute("client",myname,PageContext.SESSION_SCOPE); %>
<a href="PageContext2.jsp"> go to second jsp page</a>
</body>
</html>
```

`PageContext2.jsp`

```
<html>
<body>
```



```
<%
String
myname=(String)pageContext.getAttribute("client",PageContext.SESSION_SCOPE);
out.print("Here you will see your name "+myname);
%>
</body>
</html>
```

8. page

This implicit object comes under java.lang.Object class. Its work is to provide reference to the servlet class generated. Type casting is thus required to access this object as shown below.

```
<% (HttpServletRequest)page.log("message"); %>
```

Syntax:

```
Object page=this;
<% this.log("message"); %>
```

Example:

```
<html>
<head><title>Object page</title>
</head>
<body>
<%
this.log("message");
%>
</body>
</html>
```

JSP - Scripting Elements

JSP scripting elements are one of the most vital elements of a JSP code. These `<% %>` tags contain JSP scripting elements. These tags are of the utmost importance as, at the time of translation, the JSP engine will consider anything inside these tags. Only this code will convert to Java code. Code other than this is plain or HTML text. The scripting elements thus help to embed java code to the HTML, CSS, JavaScript code.

Here are five different scripting elements:

- Comment `<%-- Set of comment statements --%>`
- Directive `<%@ directive %>`
- Declaration `<%! declarations %>`
- Scriptlet `<% scriptlets %>`
- Expression `<%= expression %>`

1. Comments



Comments are marked as text or statements that are ignored by the JSP container. They are useful when you want to write some useful information or logic to remember in the future.

```
<%@ page import="java.util.Date"%>
<!DOCTYPE html>
<html>
  <head>
    <title>Comments in JSP Page</title>
  </head>
  <body>
    <%-- A JSP comment --%>
    <!-- An HTML comment -->
    <!--The current date is <%= new Date() %>
    -->
  </body>
</html>
```

2. Directive

These tags are used to provide specific instructions to the web container when the page is translated. It has three subcategories:

- Page: <%@ page ... %>
- Include: <%@ include ... %>
- Taglib: <%@ taglib ... %>

3. Declaration

As the name suggests, it is used to declare methods and variables you will use in your Java code within a JSP file. According to the rules of JSP, any variable must be declared before it can be used.

Syntax:

```
<%! declaration; [ declaration; ]+ ... %>
```

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <%! int variable_value=62; %>
    <%= " Your integer data is :" + variable_value %>
  </body>
</html>
```

4. scriptlet

The scriptlet tag allows writing Java code statements within the JSP page. This tag is responsible for implementing the functionality of `_jspService()` by scripting the java code.

Syntax:

```
<% JAVA CODE %>
```

Example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Scriptlet Tag</title>
  </head>
  <% int count = 2; %>
  <body>
    Calculated page count <% out.println(cnt); %>
  </body>
</html>
```

5. Expressions

Expressions elements are responsible for containing scripting language expression, which gets evaluated and converted to Strings by the JSP engine and is meant to the output stream of the response. Hence, you are not required to write out.print() for writing your data. This is mostly used for printing the values of variables or methods in the form of output.

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <%= "A JSP based string" %>
  </body>
</html>
```

JSP Scope and EL (Expression Language)

JSP Scopes

Scope decides the accessibility of an object or variable. For example, some variables are accessible within for loop, if-else block or within specific method or class or package.

Same way, in JSP some variables needs to have different scopes than others. JSP provides the capability to user to define the scope of these variables.

Type of Scopes in JSP:

JSP provides 4 scopes to a variable. Developer can assign any one of them to a variable.

1. Page Scope.
2. Request Scope.



3. Session Scope.
4. Application Scope.

Page Scope

Page Scope makes variable available to the developer for the current page only. Once the current page is closed by user or forwarded internally by application or redirected by application, than the variables having page scope will not be accessible on next page.

Example:

```
<html>
<head>
<title>JSP Page Scope Example</title>
</head>
<body>
  <c:set var="name" value="Dinesh" scope="page" />
  Local Variable : <c:out value="${name}" />
  <a href="test.jsp">Test Page</a>
</body>
</html>
```

Output :

Local Variable: Dinesh

Test Page

Second JSP File(test.jsp):

```
<html>
<head>
  <title>JSP Page Scope Example</title>
</head>
<body>
  Variable From previous page : <c:out value="${name}" />
</body>
</html>
```

Output: Variable from previous page:

Request Scope

Request Scope makes variable available to the developer for the current request only. Once the current request is over, than the variables having request scope will not be accessible on next request. Single request may include multiple pages using forward.

Example:

```
<html>
<head>
<title>JSP Request Scope Example</title>
</head>
<body>
```



```
<c:set var="name" value="Dinesh" scope="request" />
<jsp:forward page="test.jsp"></jsp:forward>
</body>
</html>
```

Second JSP File(test.jsp):

```
<html>
<head>
  <title>JSP Request Scope Example</title>
</head>
<body>
  Variable From previous page : <c:out value="{name}" />
</body>
</html>
```

Output: Variable from previous page: Dinesh

Session Scope

Session Scope makes variable available to the developer for the current session only. Once the current session is over or timed out, then the variables having session scope will not be accessible on next session.

Example:

```
<html>
<head>
<title>JSP Session Scope Example</title>
</head>
<body>
  <c:set var="name" value="Dinesh" scope="session" />
  Local Variable : <c:out value="{name}" />
  <a href="test.jsp">Test Page</a>
</body>
</html>
```

Output:

Local Variable: Dinesh
Test Page

Second JSP File(test.jsp):

```
<html>
<head>
  <title>JSP Session Scope Example</title>
</head>
<body>
  Variable From previous page : <c:out value="{name}" />
</body>
</html>
```

Output: Variable from previous page: Dinesh



Application Scope

Application Scope makes variable available to the developer for the full application. It remains available till application is running on server.

Example:

```
<html>
<head>
<title>JSP Application Scope Example</title>
</head>
<body>
  <c:set var="name" value="Dinesh" scope="application" />
  Local Variable : <c:out value="{name}" />
  <a href="test.jsp">Test Page</a>
</body>
</html>
```

Output:

Local Variable: Dinesh
Test Page

Second JSP File(test.jsp):

```
<html>
<head>
  <title>JSP Application Scope Example</title>
</head>
<body>
  Variable From previous page : <c:out value="{name}" />
</body>
</html>
```

Output: Variable from previous page: Dinesh

JSP - Expression Language (EL)

Expression Language (EL) was introduced in JSP 2.0 as a simpler way to access and manage data in JSP pages. It provides an easy syntax to retrieve values from commonly used objects like request, session, application and JavaBeans.

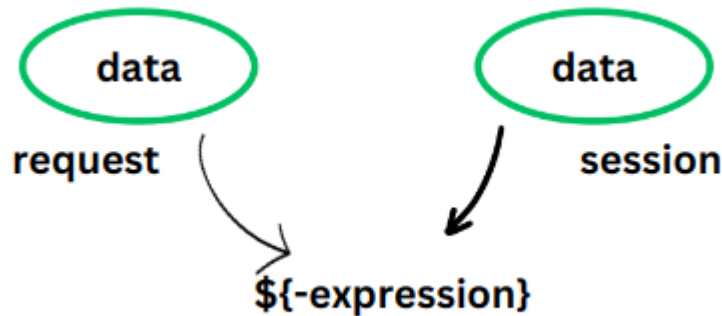
Normally, to get data from these objects, we write

1. To get data from a request, we write.
`request.getParameter("name")`
2. To get data from a session, we write.
`session.getAttribute("user")`

This makes the code longer and harder to read. But with Expression Language, we can do the same in a much simpler way:

`${param.name}`

`${sessionScope.user}`



Why Expression Language (EL)?

1. Expression Language makes the data access work much easier.
2. The data stored in the objects can be directly retrieved with the help of the Expression Language
3. The data is stored in the JavaBeans Component.
4. It allows to access a bean by following a short syntax, It can also include literals.

EL Syntax

It represents a placeholder which represents that the value of the specified *expression* should be dynamically inserted at runtime

`${expression}`

Common Usage Patterns

1. Variable Access:

Used to access the value of a variable

`${variableName}`

`${xxxScope.variableName}`

2. Property Access:

Used to access the value of a property

`${object.property}`

3. Arithmetic and Logical Operators:

Performs arithmetic or logical operations on numbers

`${num1>num2}`

`${num1+num2}`

4. Conditional Expressions:

Evaluate a condition and return one of two values.

`${condition ? value1 : value2 }`

5. Collection Iterator :

Iterates over a collection and prints each item



```
<c:forEach var="item" items = "${collection}" > ${item} </c:forEach >
```

6. Accessing Request Parameters:

Accesses the value of a request parameter

```
${param. parametersName}
```

Types of Expression Language

1. Request Object

This code snippet demonstrates how to access and display the value of a request attribute using Expression Language (EL). It sets the request attribute request_name to "GeeksforGeeks" and then displays its value using EL within an HTML element.

Example:

```
<html>
<head>
<title>Expression Language</title>
</head>
<body>
<% request.setAttribute("request_name", "CIT"); %>
<h3>${requestScope.request_name}</h3>
</body>
</html>
```

Output:

CIT

2. Using the instanceof session as object

This code snippet showcases how to access and display the value of a session attribute using EL. It sets the session attribute request_name to "GeeksforGeeks" and then displays its value using EL within an HTML element.

Example:

```
<html>
<head>
<title>Expression Language</title>
</head>
<body>
<% request.setAttribute("request_name", "CIT"); %>
<h3>${requestScope.request_name} </h3>
</body>
</html>
```

output:

CIT

Some implicit objects in Expression Language are:

1. **requestScope:** contains attributes that are specific to the current HTTP request.
2. **param:** provides access to request parameters sent to the server.



3. **paramValues:** provides access to request parameter values as an array.
4. **sessionScope:** Contains attributes that are specific to the current user session.
5. **pageContent:** Provides access to various objects and methods related to the JSP page content.
6. **pageScope:** Contains attributes that are specific to the current JSP Page.
7. **applicationScope:** Contains attributes that are specific to the entire web application, etc.

Expression Language also works for

- Arithmetic Operators (+, -, *, /, %)
- Logical Operators (&&, ||, !)
- Relational Operators (==, !=, >, <, <=, >=)
- Conditional Operator (?)
- Empty Operator (empty, used to check if a value is empty, such as null or an empty collection/string)

3. Using Expression Language in forms

Arithmetic Expression

This code snippet demonstrates the use of EL for performing arithmetic operations. It evaluates the expression `10+90` and displays the result inline within an HTML element.

Example:

```
<html>
<head>
  <title>Page Title</title>
</head>
<body>
  ${10+90}
</body>
</html>
```

output:

```
100
```

JSP Standard Tag Libraries (JSTL)

JSTL (JavaServer Pages Standard Tag Library) is a collection of custom tags that encapsulate core functionalities commonly used in JSP pages. It helps reduce Java code embedded in JSPs (known as scriptlets) and promotes a cleaner MVC-based web application design. JSTL is part of the Jakarta EE (formerly Java EE) specification and is supported in all modern servlet containers.

Classification of The JSTL Tags

The JSTL tags can be classified, according to their functions, into the following JSTL tag library groups that can be used when creating a JSP page –

1. Core Tags
2. Formatting tags
3. SQL tags
4. XML tags



5. JSTL Functions

1.Core Tags

The core group of tags are the most commonly used JSTL tags. Following is the syntax to include the JSTL Core library in your JSP –

```
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
```

Following table lists out the core JSTL Tags –

S.No.	Tag & Description
1	<c:out> Like <%= ... >, but for expressions.
2	<c:set > Sets the result of an expression evaluation in a 'scope'
3	<c:remove > Removes a scoped variable (from a particular scope, if specified).
4	<c:catch> Catches any Throwable that occurs in its body and optionally exposes it.
5	<c:if> Simple conditional tag which evaluates its body if the supplied condition is true.
6	<c:choose> Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by <when> and <otherwise>.
7	<c:when> Subtag of <choose> that includes its body if its condition evaluates to 'true'.
8	<c:otherwise >

	Subtag of <choose> that follows the <when> tags and runs only if all of the prior conditions evaluated to 'false'.
9	<c:import> Retrieves an absolute or relative URL and exposes its contents to either the page, a String in 'var', or a Reader in 'varReader'.
10	<c:forEach > The basic iteration tag, accepting many different collection types and supporting subsetting and other functionality .
11	<c:forTokens> Iterates over tokens, separated by the supplied delimiters.
12	<c:param> Adds a parameter to a containing 'import' tag's URL.
13	<c:redirect > Redirects to a new URL.
14	<c:url> Creates a URL with optional query parameters

2.Formatting Tags

The JSTL formatting tags are used to format and display text, the date, the time, and numbers for internationalized Websites. Following is the syntax to include Formatting library in your JSP –

```
<%@ taglib prefix = "fmt" uri = "http://java.sun.com/jsp/jstl/fmt" %>
```

Following table lists out the Formatting JSTL Tags –

S.No.	Tag & Description
1	<fmt:formatNumber> To render numerical value with specific precision or format.

2	<code><fmt:parseNumber></code> Parses the string representation of a number, currency, or percentage.
3	<code><fmt:formatDate></code> Formats a date and/or time using the supplied styles and pattern.
4	<code><fmt:parseDate></code> Parses the string representation of a date and/or time
5	<code><fmt:bundle></code> Loads a resource bundle to be used by its tag body.
6	<code><fmt:setLocale></code> Stores the given locale in the locale configuration variable.
7	<code><fmt:setBundle></code> Loads a resource bundle and stores it in the named scoped variable or the bundle configuration variable.
8	<code><fmt:timeZone></code> Specifies the time zone for any time formatting or parsing actions nested in its body.
9	<code><fmt:setTimeZone></code> Stores the given time zone in the time zone configuration variable
10	<code><fmt:message></code> Displays an internationalized message.
11	<code><fmt:requestEncoding></code> Sets the request character encoding

3.SQL Tags

The JSTL SQL tag library provides tags for interacting with relational databases (RDBMSs) such as Oracle, MySQL, or Microsoft SQL Server.

Following is the syntax to include JSTL SQL library in your JSP –

```
<%@ taglib prefix = "sql" uri = "http://java.sun.com/jsp/jstl/sql" %>
```



Following table lists out the SQL JSTL Tags –

S.No.	Tag & Description
1	<code><sql:setDataSource></code> Creates a simple DataSource suitable only for prototyping
2	<code><sql:query></code> Executes the SQL query defined in its body or through the sql attribute.
3	<code><sql:update></code> Executes the SQL update defined in its body or through the sql attribute.
4	<code><sql:param></code> Sets a parameter in an SQL statement to the specified value.
5	<code><sql:dateParam></code> Sets a parameter in an SQL statement to the specified java.util.Date value.
6	<code><sql:transaction ></code> Provides nested database action elements with a shared Connection, set up to execute all statements as one transaction.

4.XML tags

The JSTL XML tags provide a JSP-centric way of creating and manipulating the XML documents. Following is the syntax to include the JSTL XML library in your JSP.

The JSTL XML tag library has custom tags for interacting with the XML data. This includes parsing the XML, transforming the XML data, and the flow control based on the XPath expressions.

```
<%@ taglib prefix = "x" uri = "http://java.sun.com/jsp/jstl/xml" %>
```

Following is the list of XML JSTL Tags –

S.No.	Tag & Description
1	<code><x:out></code> Like <code><%= ... ></code> , but for XPath expressions.

2	<x:parse> Used to parse the XML data specified either via an attribute or in the tag body.
3	<x:set > Sets a variable to the value of an XPath expression.
4	<x:if > Evaluates a test XPath expression and if it is true, it processes its body. If the test condition is false, the body is ignored.
5	<x:forEach> To loop over nodes in an XML document.
6	<x:choose> Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by <when> and <otherwise> tags.
7	<x:when > Subtag of <choose> that includes its body if its expression evaluates to 'true'.
8	<x:otherwise > Subtag of <choose> that follows the <when> tags and runs only if all of the prior conditions evaluates to 'false'.
9	<x:transform > Applies an XSL transformation on a XML document
10	<x:param > Used along with the transform tag to set a parameter in the XSLT stylesheet

Example of JSTL

```

<%@ taglib uri="http://java.sun.com/jstl/core" prefix="c" %>
<html>
<head>
<title> Numbers</title>
</head>

<body>
<c:forEach var="i" begin="1" end="5">
  Value<c:out value="${i}" />

```

```
<br />  
</c:forEach>  
</body>  
</html>
```

Output: The output will be as follows:

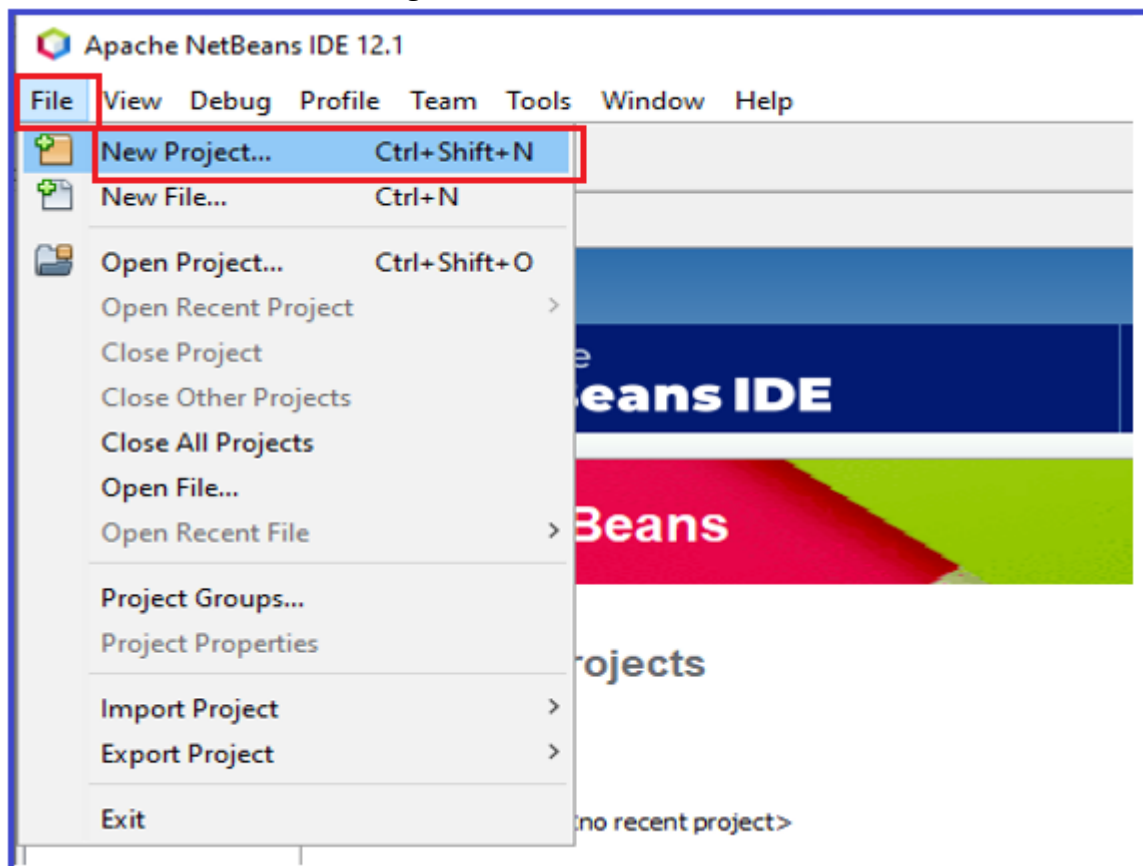
Value1
Value2
Value3
Value4
Value5

Developing Dynamic Web Applications with JSP

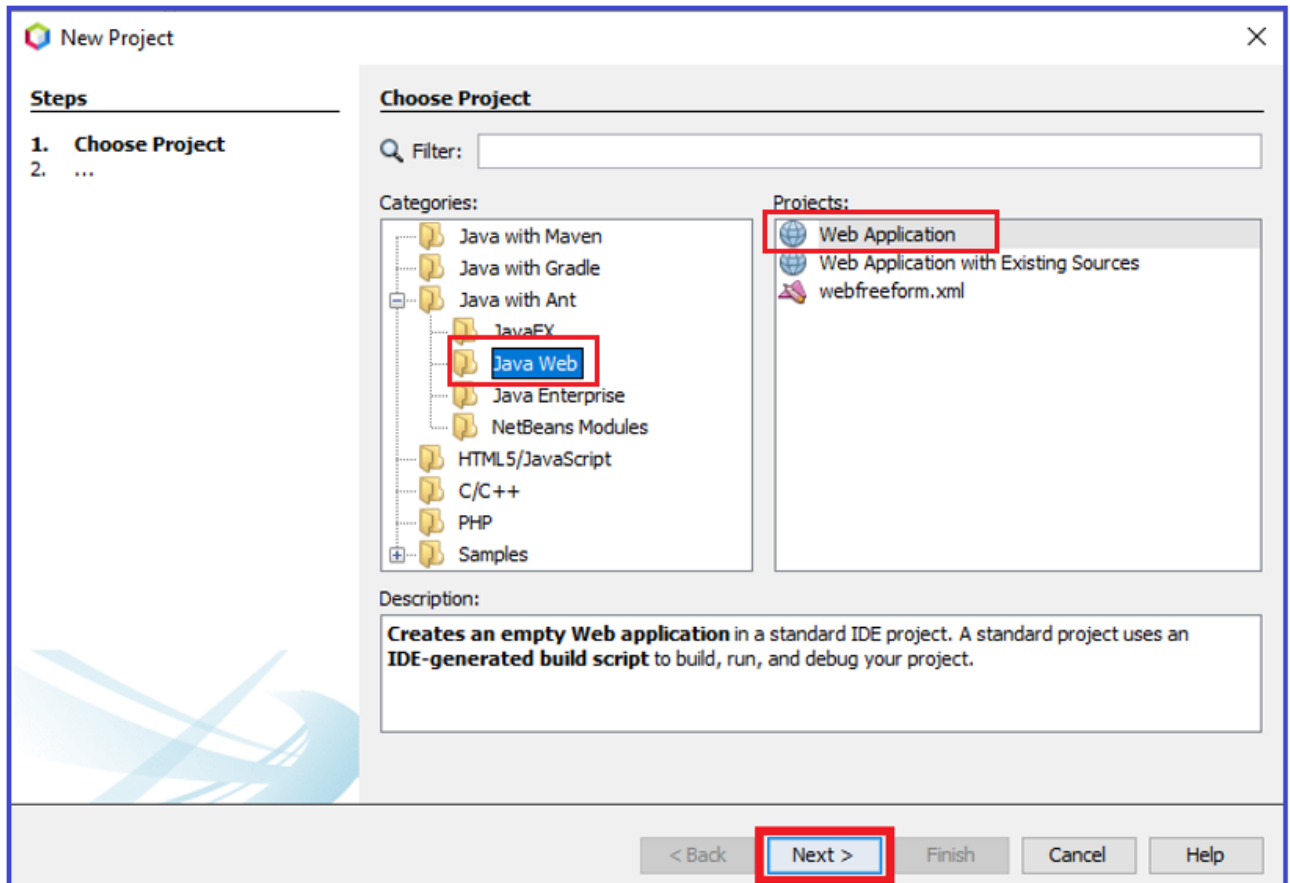
We will create a simple JSP file and run it on NetBeans IDE using Apache Tomcat Server. Follow the below steps to create the servlet in the NetBeans IDE:

Step-1: Create New Project

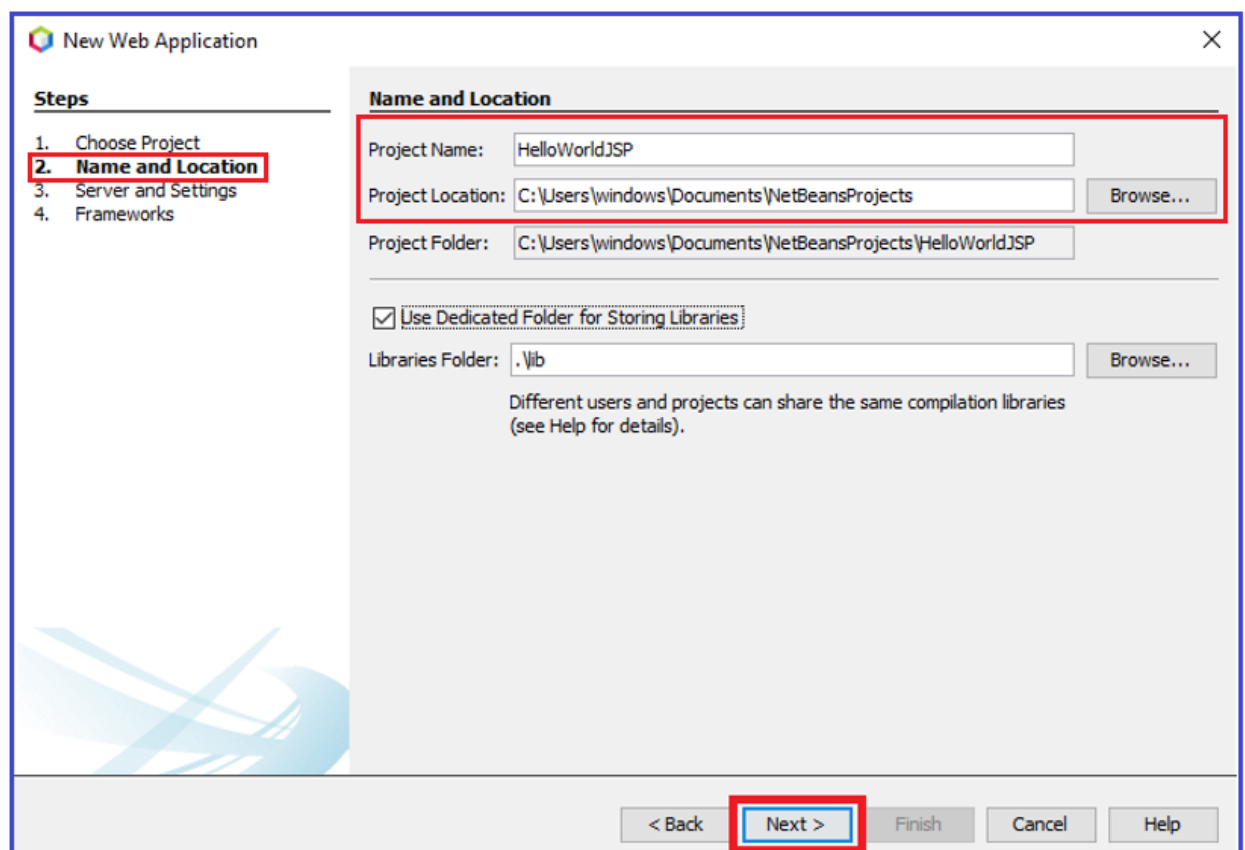
- Open NetBeans IDE and then select File -> New Project option from the context menu as shown in the below image.



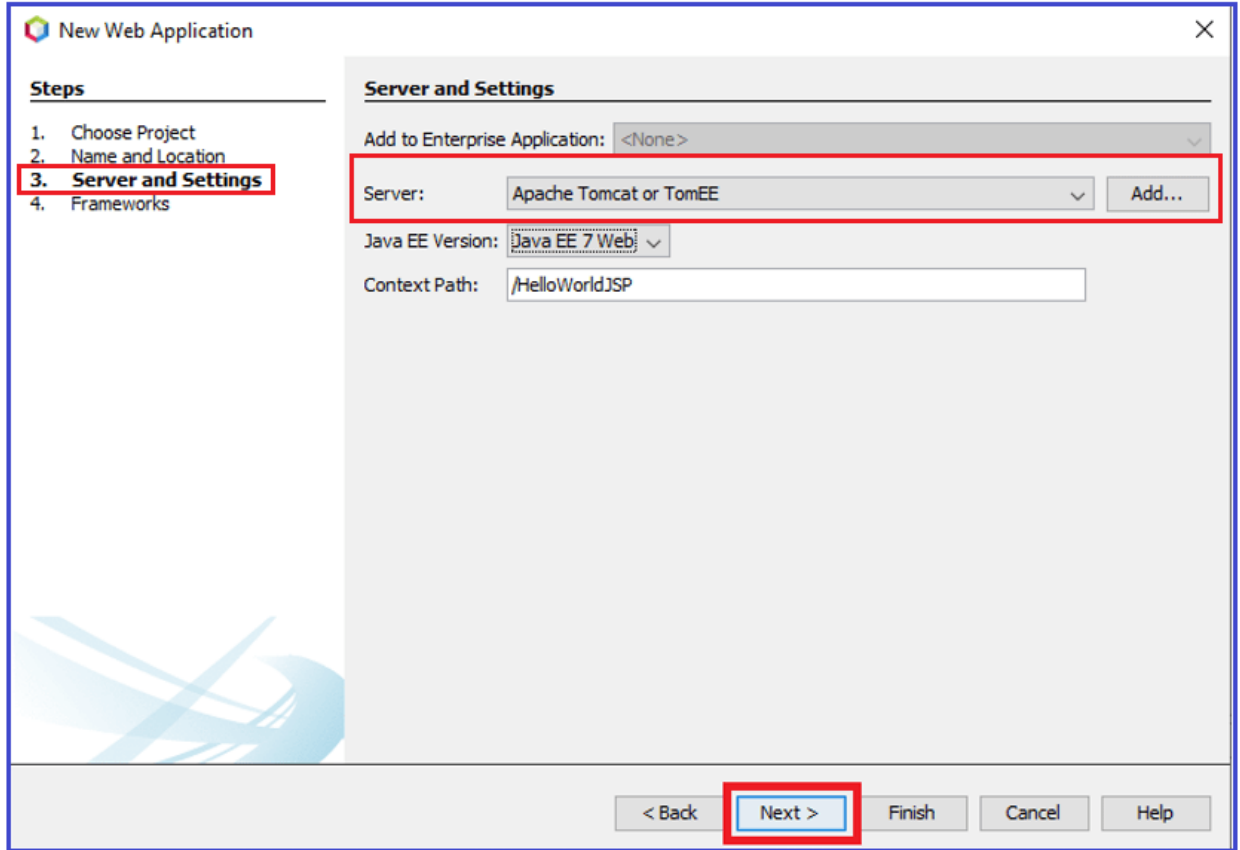
- It will open the New Project window. From this window, select Java web -> Web Application and then click on the Next button as shown in the below image.



- Once you click on the Next button, a new window is generated with a default project name.
- Now change the project name to “HelloWorldJSP” and select the location where you want to create the project and then click on the “Next” button as shown in the below image.



- Once you click on the Next button, a new window is generated for the server and setting the wizard. First, go to the server and click on “Add server”.
- Since I already set up the Tomcat Server, it is shown there but when you do it the first time it shows blank. And then click on the “Next” button as shown in the below image.



New Web Application

Steps

1. Choose Project
2. Name and Location
- 3. Server and Settings**
4. Frameworks

Server and Settings

Add to Enterprise Application: <None>

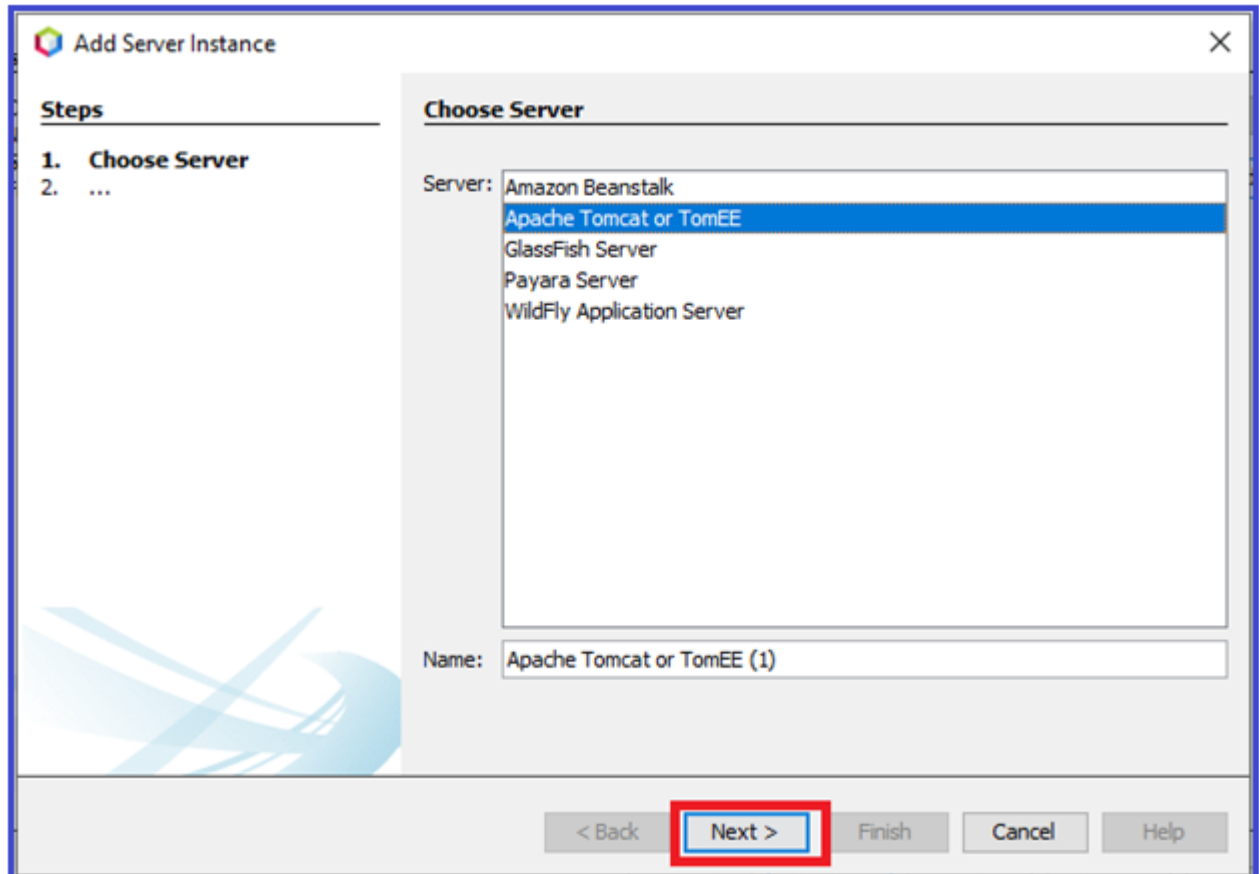
Server: Apache Tomcat or TomEE **Add...**

Java EE Version: Java EE 7 Web

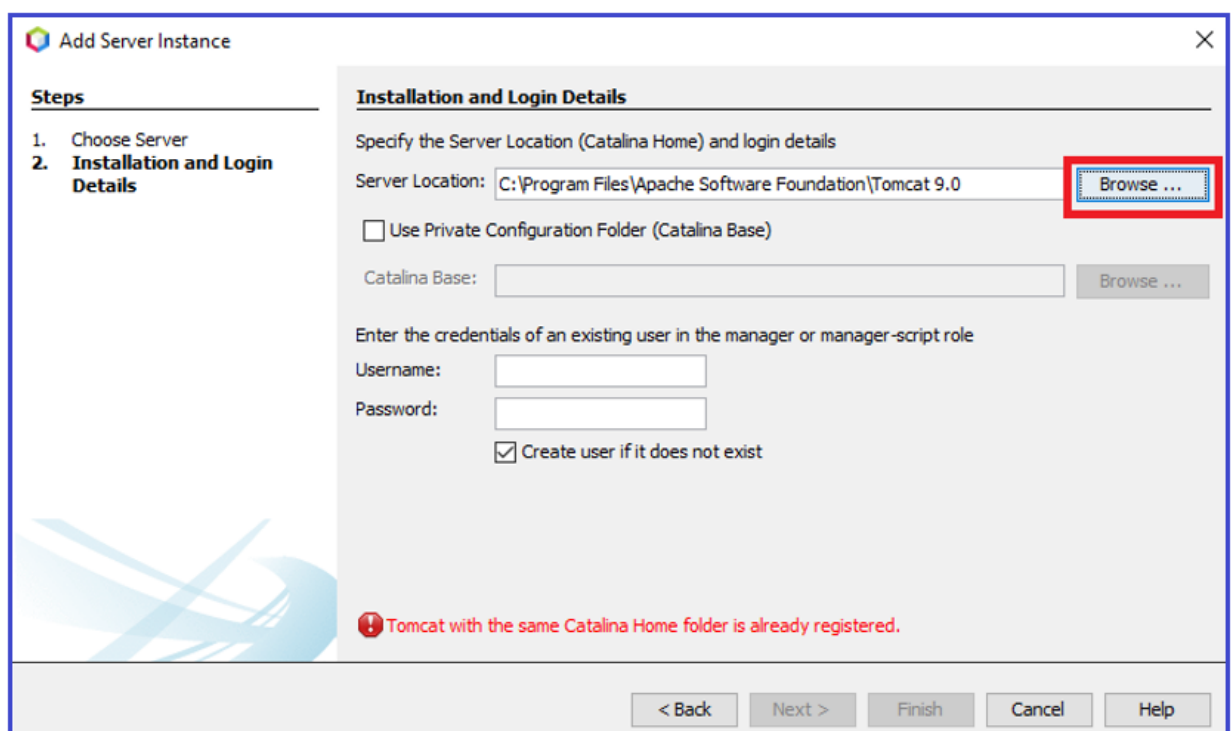
Context Path: /HelloWorldJSP

< Back **Next >** Finish Cancel Help

- After clicking on Add, a list of servers opens. In this article, I used Tomcat Server 7.0.40. Choose your server and configure it. For Tomcat Server choose it and click on “Next”.



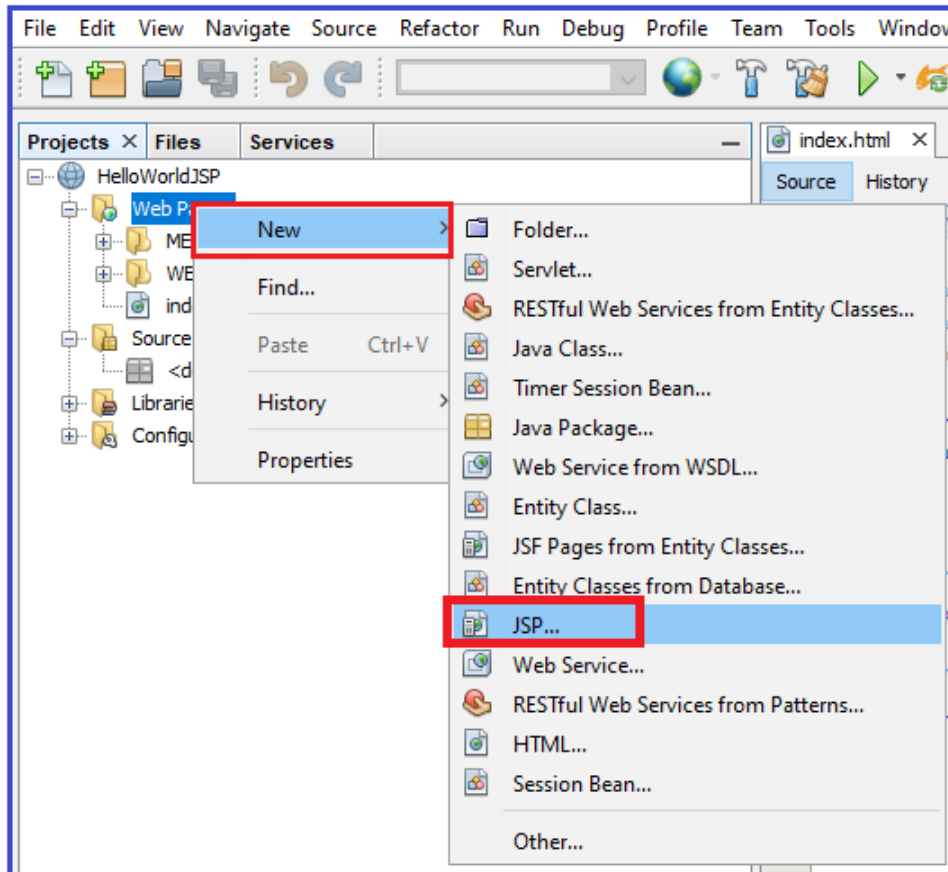
-
- Now a window opens asking for the server location. Now, click on the Browse button and find the server location where you have stored the Apache Tomcat. Note that you need to provide the path up to the bin directory of Tomcat Server.
- After providing the server path you will see two more options for username and password. It's your choice whether you want your username and password or you want the default created by the Netbeans IDE. Now click on "Finish".



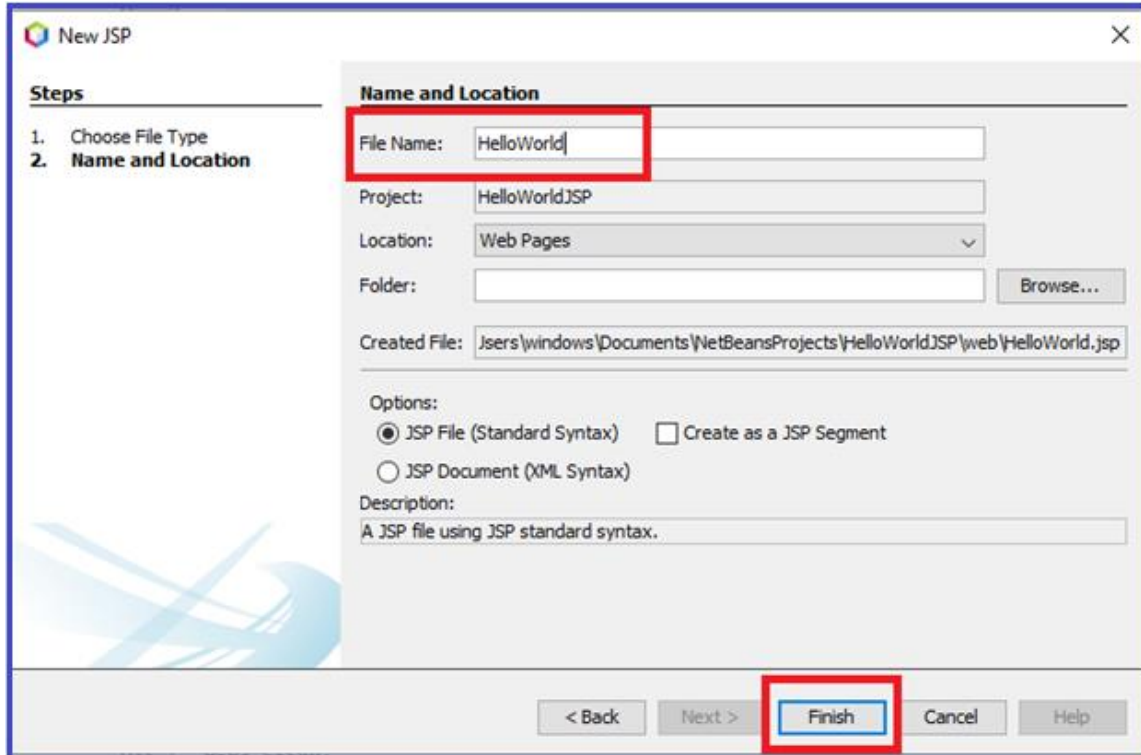
- Now a window is generated containing the default index.html file in Web Pages under HelloWorldJSP.

Step-2: Creating JSP Page

- In order to create a JSP Page, right-click on your project, and then select **New -> JSP** option as shown in the below image.



- It will open the New JSP window. Here, give the File Name as “HelloWorld” and then click on the “Finish” button as shown below.



New JSP

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

File Name:

Project:

Location:

Folder:

Created File:

Options:

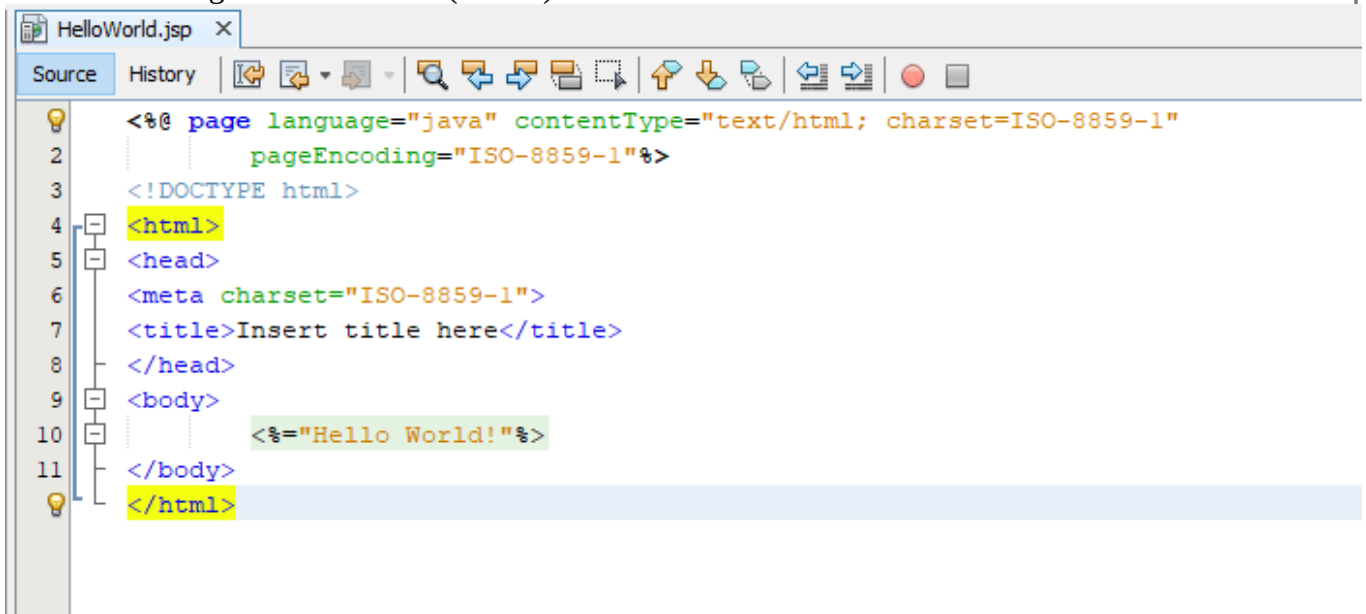
☒ JSP File (Standard Syntax) ☐ Create as a JSP Segment

☐ JSP Document (XML Syntax)

Description:

A JSP file using JSP standard syntax.

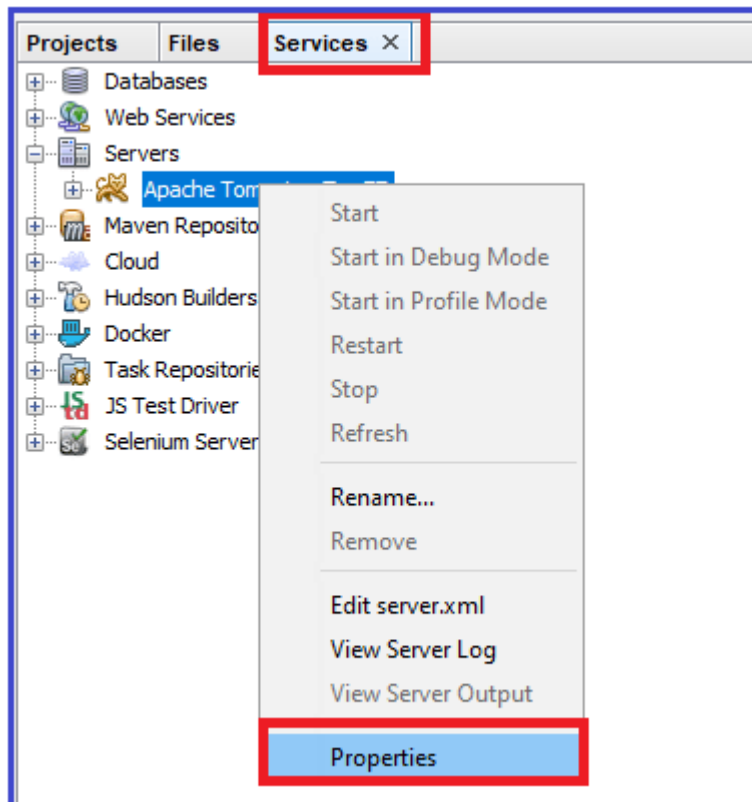
- Once you click on the “Finish” button a JSP page will create automatically where edit the following code and save it (Ctrl+S).



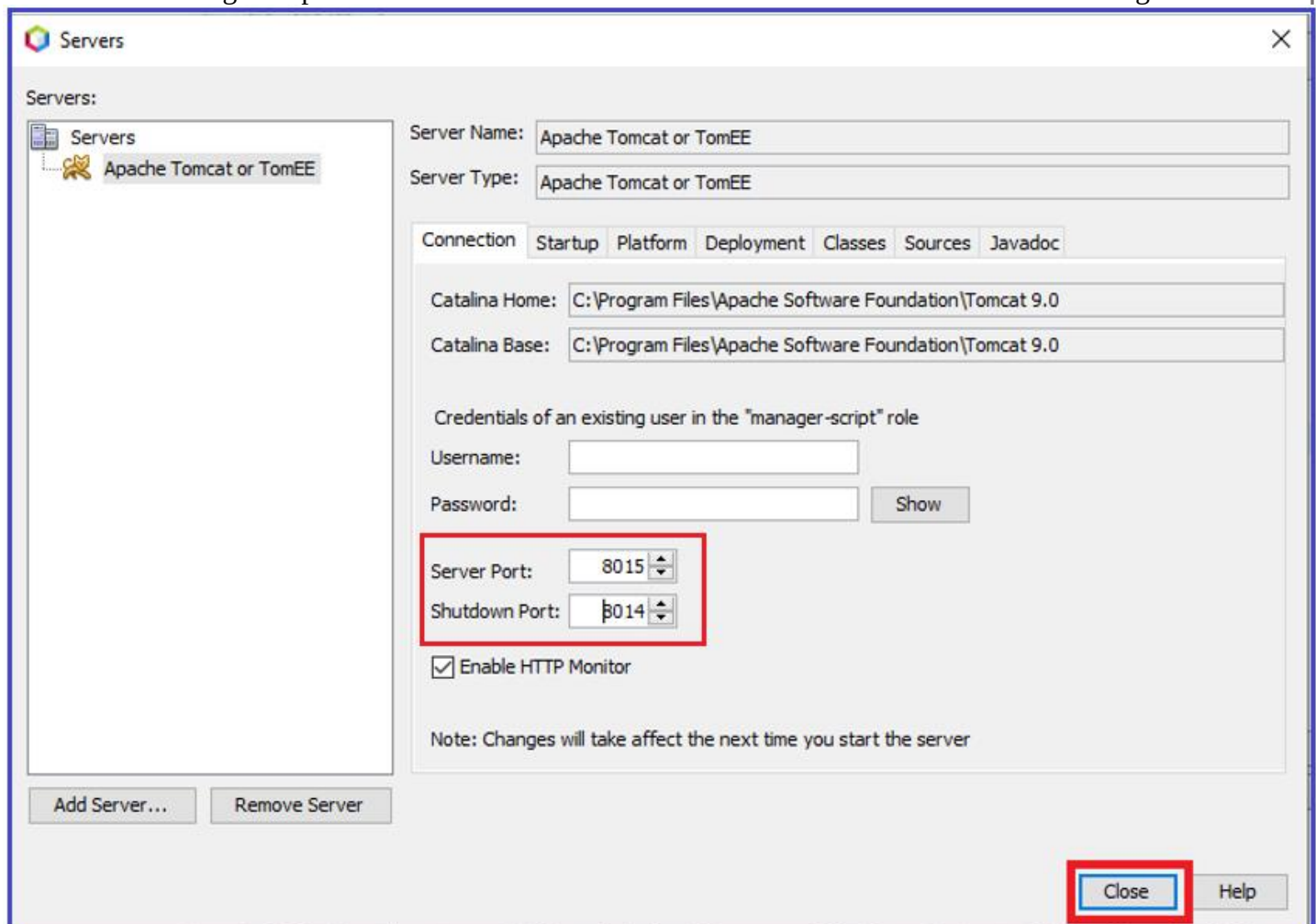
```

1  <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2      pageEncoding="ISO-8859-1"%>
3  <!DOCTYPE html>
4  <html>
5  <head>
6  <meta charset="ISO-8859-1">
7  <title>Insert title here</title>
8  </head>
9  <body>
10     <%= "Hello World!" %>
11 </body>
12 </html>
  
```

- For changing the port number for your server, go to service right click on Apache Tomcat -> Properties

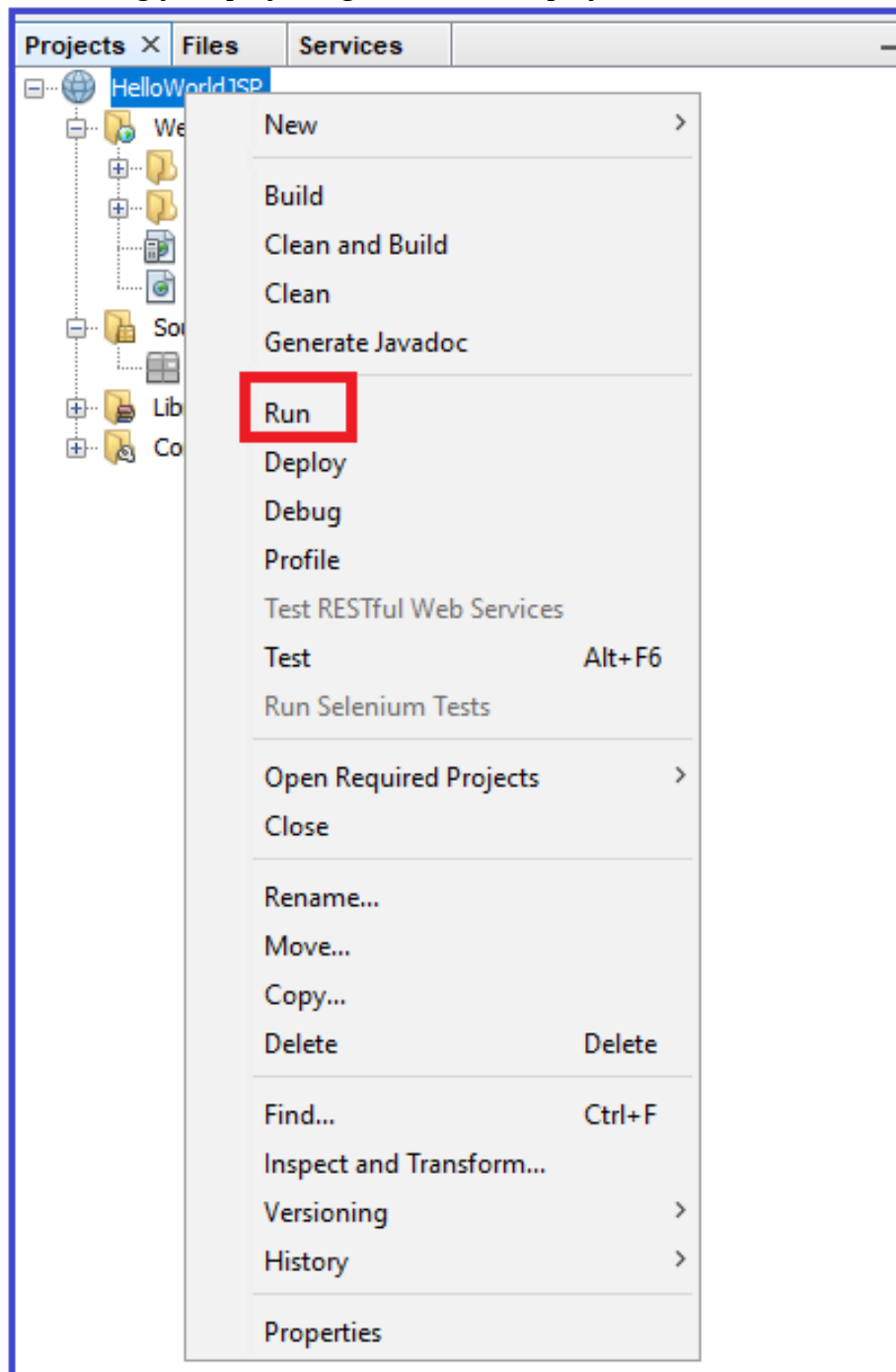


- Change the port number and click on the “close” button as shown in the below image.



Step-3: Run your Project

- For running your project right-click on the project and select “Run” as follows.



- The following output is generated through the specified JSP file.

